

Task 2 Documentation

API Integration & JSON Handling

Name: Harshitha Bolla

Course: B.Tech CSE, Anurag University

Internship: Alfido Tech - Python Internship

Task: Task 2 - API Integration & JSON Handling

1. Objective

The objective of this task was to understand how Python communicates with external APIs and handles JSON data. The script sends a request to a public API, parses the JSON response, applies search/filtering logic based on user input, and handles errors that may occur during the API call.

2. Requirements Covered

- Fetch data using the Requests library
- Parse JSON responses
- Apply filtering or search logic
- Handle API errors

3. Tools Used

- Python 3
- requests module
- JSONPlaceholder API (jsonplaceholder.typicode.com/users)

4. Project Structure

The task2 folder contains the script (api_project.py), a README file, and a screenshots folder used to store output screenshots.

5. Key Code Logic

The full script is available in the GitHub repository (link at the end of this document). Below are the main logic parts of the code.

Sending the request and parsing the JSON response:

```
import requests

url = "https://jsonplaceholder.typicode.com/users"

try:
    response = requests.get(url, timeout=5)
```

```
response.raise_for_status()
data = response.json()
```

Searching for the user by username:

```
search_name = input("Enter username to search: ")
found = False

for user in data:
    if user["username"].lower() == search_name.lower():
        print("\nUser Found")
        print("Name:", user["name"])
        print("Email:", user["email"])
        print("City:", user["address"]["city"])
        found = True
```

Handling the case where no user matches:

```
if not found:
    print("User not found")
```

Handling API errors:

```
except requests.exceptions.RequestException as e:
    print("Error while fetching API data:", e)
```

6. Code Explanation

- Setup: The script imports the requests module and defines the API URL pointing to JSONPlaceholder's /users endpoint.
- Sending the Request: requests.get() sends a GET request with a timeout of 5 seconds, and response.raise_for_status() raises an exception automatically if the server returns an error status code.
- Parsing JSON: response.json() converts the raw JSON response into a Python list of dictionaries, one dictionary per user.
- Search Logic: The script asks the user for a username, then loops through the list of users and compares each user's username field (case-insensitive) with the input.
- Match Found: If a match is found, the user's Name, Email, and City are printed under a "User Found" heading, and a found flag is set to True.
- No Match Found: If the loop completes without a match, the found flag remains False and the script prints "User not found."

7. Exception Handling

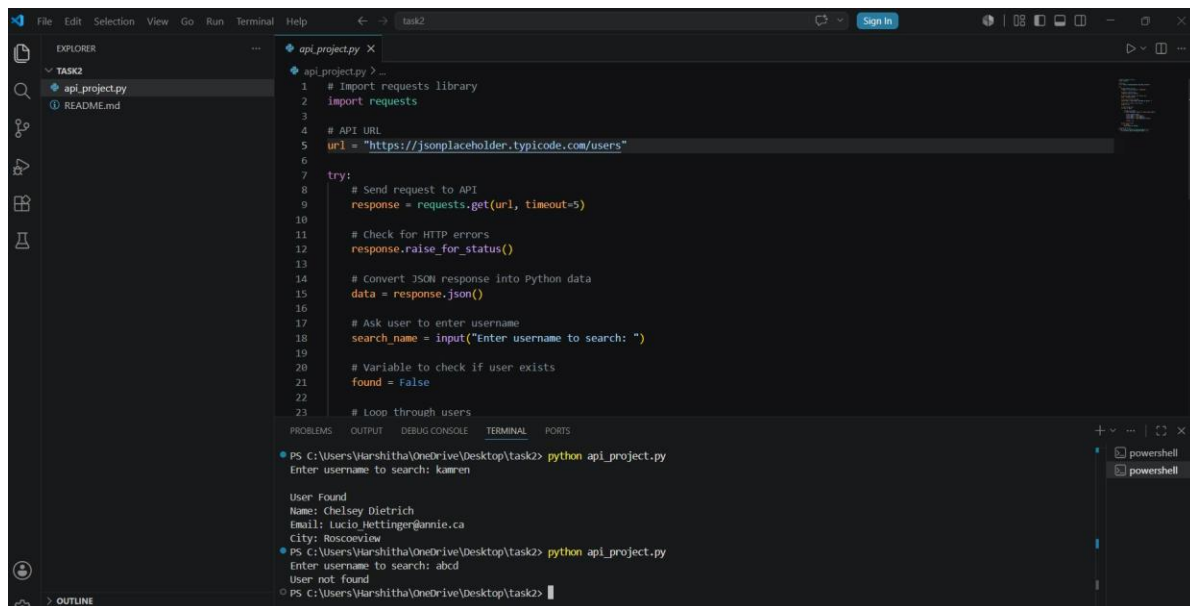
The script uses a try-except block around the API call and data processing to avoid the program crashing if an error occurs. Some examples:

- `ConnectionError` - if the URL is incorrect or the server cannot be reached
- `Timeout` - if the server takes too long to respond
- `HTTPError` - if `raise_for_status()` detects a bad response code (e.g. 404 or 500)

All of these are subclasses of `requests.exceptions.RequestException`, which is caught in the except block. This ensures that instead of crashing, the script prints a clear error message describing what went wrong.

8. Sample Input and Output

Valid and Invalid Username: When a valid username (e.g. “kamren”) is entered, the script prints “User Found” along with the matching user's Name, Email, and City. When an invalid username (e.g. “abcd”) is entered, the script prints “User not found.”



The screenshot shows a Visual Studio Code window with a Python file named `api_project.py` and a terminal window. The script in the editor uses the `requests` library to fetch data from an API. It includes a try-except block to handle potential errors. The terminal shows the execution of the script, where a valid username 'kamren' results in 'User Found' and an invalid username 'abcd' results in 'User not found'.

```
1 # Import requests library
2 import requests
3
4 # API URL
5 url = "https://jsonplaceholder.typicode.com/users"
6
7 try:
8     # Send request to API
9     response = requests.get(url, timeout=5)
10
11     # Check for HTTP errors
12     response.raise_for_status()
13
14     # Convert JSON response into Python data
15     data = response.json()
16
17     # Ask user to enter username
18     search_name = input("Enter username to search: ")
19
20     # Variable to check if user exists
21     found = False
22
23     # Loop through users
```

Terminal Output:

```
PS C:\Users\Varshitha\OneDrive\Desktop\task2> python api_project.py
Enter username to search: kamren

User Found
Name: Chelsey Dietrich
Email: lucio.hettinger@annie.ca
City: Roscoeview

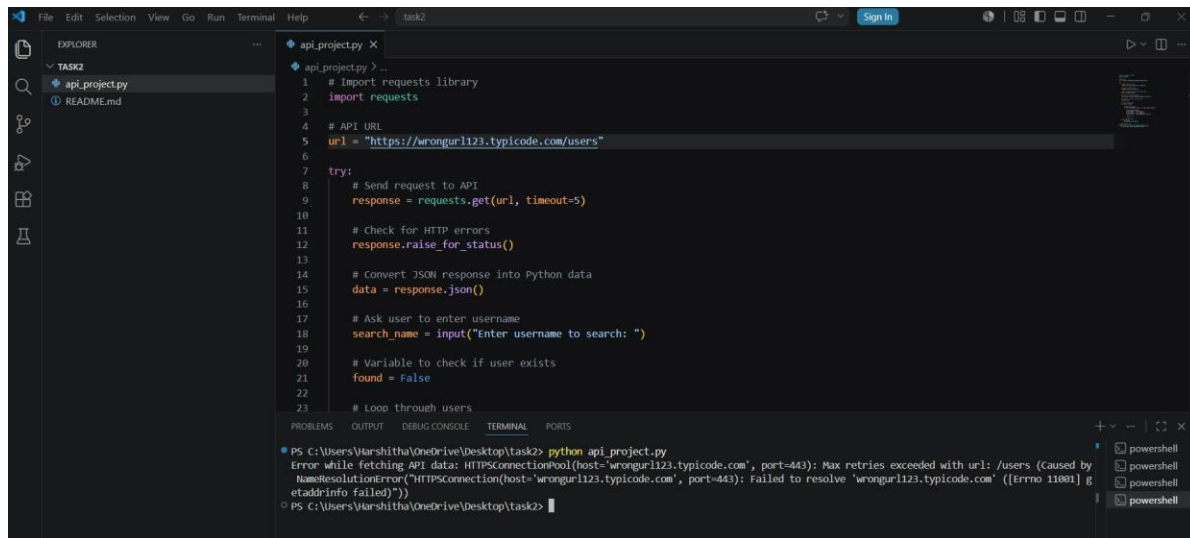
PS C:\Users\Varshitha\OneDrive\Desktop\task2> python api_project.py
Enter username to search: abcd

User not found

PS C:\Users\Varshitha\OneDrive\Desktop\task2>
```

Figure 1: Output for a valid username (User Found) and an invalid username (User not found)

API Error - Invalid URL: When the API URL is incorrect or unreachable, `requests.get()` raises a connection error. This is caught by the except block, which prints “Error while fetching API data:” followed by the exception details instead of letting the program crash.



The image shows a Visual Studio Code window with a Python file named `api_project.py` open. The file contains the following code:

```
1 # Import requests library
2 import requests
3
4 # API URL
5 url = "https://wrongurl123.typicode.com/users"
6
7 try:
8     # Send request to API
9     response = requests.get(url, timeout=5)
10
11     # Check for HTTP errors
12     response.raise_for_status()
13
14     # Convert JSON response into Python data
15     data = response.json()
16
17     # Ask user to enter username
18     search_name = input("Enter username to search: ")
19
20     # Variable to check if user exists
21     found = False
22
23     # Loop through users
```

The terminal at the bottom shows the command `python api_project.py` being executed. The output is an error message:

```
PS C:\Users\Varshitha\OneDrive\Desktop\task2> python api_project.py
Error while fetching API data: HTTPConnectionPool(host='wrongurl123.typicode.com', port=443): Max retries exceeded with url: /users (caused by NameResolutionError("HTTPConnection(host='wrongurl123.typicode.com', port=443): Failed to resolve 'wrongurl123.typicode.com' ([Errno 11001] getaddrinfo failed)))
PS C:\Users\Varshitha\OneDrive\Desktop\task2>
```

Figure 2: Output when the API URL is invalid

9. Conclusion

This task helped in understanding how Python can be used to communicate with external APIs and process JSON data. It covered sending GET requests, parsing JSON responses, applying search/filter logic based on user input, and handling errors so the program does not crash when the API or network fails.

10. GitHub Repository

<https://github.com/HarshithaBolla17/Alfido-Tech-Python-Internship>